# Introduktion till C++

## Uppgift 4 – Diablo

### Uppgiftens målsättning

Uppgiften utgår från nuvarande kunskapsnivå för att ge den studerande övning, erfarenhet och kunskap av att implementera ett större projekt med mer komplexitet i C++ från grunden. Den studerande får lära sig och öva på att arbeta iterativt. Design patterns som koncept introduceras och motiveras med övning där factory pattern används. Uppgiften introducerar även dynamisk collection med `std::vector`. Den studerande får även öva på att tänka kring åtkomst i minnet med pekare och referenser för att bygga förståelse för datas ägande och livslängd. Divide and conquer ges en central roll som koncept för att lösa problem och bejaka en iterativ process.

### Uppgift

Uppgiften kommer vara uppdelad i tre delmoment:

**Uppgift 4.1 – Diablo I: Dungeon**
Vi bryter ned ett textbaserat Diablo-liknande spel med ett win-condition för att kunna bygga grunden till spelet utan att fokusera på detaljer (divide and conquer). Spelaren ska kunna gå mellan olika rum med hjälp av dörrar. I vissa rum finns fiender som hindrar spelaren från att gå till nästa rum. Ett enkelt stridssystem gör att spelaren kan besegra fienderna och gå vidare.

**Uppgift 4.2 – Diablo II: Loots**
Uppgiften är att öka komplexiteten på diablo-spelet genom att introducera flertalet olika spelobjekt.

**Uppgift 4.3 – Diablo III: Hell Factory**
I sista delen av uppgiften skall ni skapa ett antal *Factory*-objekt. Dessa ska bygga instanser av de klasser som behövs i spelet vid förfrågan (då motsvarande funktioner anropas). För att uppnå detta kommer de att behöva *Type object*.

**Endast för VG: Uppgift 5 – Diablo – Inventory**

För VG implementeras inventory, spellbook och equipment som innebär högre komplexitet där koppling mellan objekt och objekts livslängd och ägande får en central roll.

TGA
THE GAME ASSEMBLY

## Inlämning

Detta är en **delmomentsuppgift** som inte behöver lämnas in.
Dock **rekommenderar** vi att ni gör det för feedback på er kod samt att redan nu komma in i hur uppgifter skall lämnas in framöver.

Skapa en mapp med **uppgiftens namn**. I denna mapp skapar ni ytterligare två mappar. En som heter **Exe** och en som heter **Source**.

**Source**-mappen innehåller all källkod samt projekt filer.
**Exe**-mappen innehåller endast filer som behövs för att köra programmet.

**Ta bort alla skräpfiler!!**

*Läs igenom ”Inlämningsinstruktioner”-dokumentet för en mer komplett genomgång om hur uppgifter skall lämnas in.*